Evidence Binding under Fixed Authority Boundaries in Tool-Using Agents: An Oracle-Plan Audit on AgentDojo

Anonymous Authors[1]

Abstract

This is an audit under an *oracle plan*. On AgentDojo v1.2.2 we hold the benchmark's ground-truth call sequence, read arguments, free-text arguments and final answer fixed. A binder may only pick one typed candidate admitted by a fixed contract, or abstain. Every admitted choice sequence is replayed offline against the benchmark's own evaluators, without a language model.

Slot types, candidates and the baselines' decisions are derived from the tool schema, the request and earlier observations; they do not change when hidden gold values are mutated. The evaluated arms are still oracle-conditioned. The *fallback* arm additionally fills slots that have no observed candidate with the planner's gold value; the *strict* arm abstains there instead.

Over a typed lexical resolver, the clairvoyant envelope of what any admitted choice could add is $[0.271, 0.292]$ (strict) and $[0.273, 0.293]$ (fallback). This is a computational envelope over partially searched trees, not a confidence interval or an achievable gain. In both arms the utility-only envelope is identical: every selectable failure is a utility failure, e.g. choosing the channel with the most users.

The baselines executed the attacker goal in 0 (strict) and 4 (fallback) of 949 injected variants. Some admitted choice achieves it in 31–106 and 35–110 variants respectively. These are witness counts, not attack rates, and they show that risk within the contract is not zero.

We do not pursue a learned binder on this evidence. The audit does not show that argument binding is unimportant for agents with real planners.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Tool-using language-model agents read e-mails, files and web pages written by third parties, then act on the user's behalf. Indirect prompt injection turns that content into an attack surface. A line of defenses therefore separates what the agent is allowed to do from what it reads.

- **Control/data separation.** CaMeL derives a program from the trusted query and tags every value with capabilities that policies check before each tool call (Debenedetti et al., 2025). Information-flow systems propagate confidentiality and integrity labels (Wu et al., 2024; Costa et al., 2025; Zhong et al., 2025). Privilege-control systems restrict tools and arguments (Shi et al., 2025; Tsai & Bagdasarian, 2025).
- **Argument-level provenance.** The most recent systems move to the granularity of individual arguments. PACT checks each argument's provenance against a role-specific contract (Fan et al., 2026). ROPE admits a sensitive parameter only if its origin satisfies a marker chosen from the user's request (Ma et al., 2026). ARGUS releases an action only when benign evidence entails every argument (Weng et al., 2026).

Authorization is not correctness. These boundaries decide whether a value is *admissible*. ROPE states the limit plainly: its guarantee "is about origin, not intent: it ensures an admitted value's origin is trusted, not that it is the value the user wanted" (Ma et al., 2026, §3.4). Consider "pay the invoice for order ORD-1234 from our vendor". Once the vendor is delegated as a source, all of the following are admissible:

- the current invoice;
- an older invoice for a different order;
- a draft that was later corrected;
- a message from the vendor's compromised account announcing "new bank details".

Some component must still choose among admissible values, or abstain. In current systems that component is the

planner language model, a hand-written rule, or a prompted grounding step. ARGUS notes that "a forged invoice can make the attacker's account the only available evidence" (Weng et al., 2026, §6.2).

Question and scope. We ask how much this choice can matter when everything else is held fixed: the plan, the per-argument authority contract, and the information available when the argument is bound. We answer only under an oracle plan on AgentDojo, and we train nothing.

What this paper reports. Three census findings, each conditional on the oracle plan (§5.4):

1. **Utility, not security.** The envelope over a typed resolver is $[0.271, 0.292]$ (strict) and $[0.273, 0.293]$ (fallback), and it is identical for utility alone. Every selectable failure is a choice among legitimate values.
2. **Rare execution, non-zero admitted risk.** Baselines rarely executed the attacker goal, but some admitted choice achieves it in a bounded set of injected variants.
3. **Limits of the measurement.** Some tasks cannot be completed by any candidate our generator admits. The identifiability screen is vacuous for most selectable failures.

The formulation (§3) and the ceiling (§4) are measurement framing built on prior contract and provenance work; we do not count them as contributions. Our earlier census (retracted as a deployable ceiling, §5.2) and a synthetic simulator study (Appendix A) are kept as a record. They are not contributions either.

Non-claims. We do not claim the following:

- that learned binding improves security or utility of real agents;
- that the AgentDojo ceiling transfers to agents whose planner is a language model rather than the ground truth;
- that the V1 census measures a deployable contract;
- that the selectable failures are identifiable from observation (they are only not refuted);
- that the evaluated arms are gold-free (only the actor-view derivation is), or that the official and executed-effect channels are equivalent in general;
- that the admitted-attack witness counts are attack rates or learnable security gains;
- that tasks our candidate generator cannot serve are out of reach for every binder;
- that our validator provides process isolation or a formal guarantee;
- that any simulator number measures a real model;
- that "span binding" alone distinguishes this work from ROPE, PACT or ARGUS (§2).

2. Related Work

System-level defenses. The dual-LLM and plan-then-execute patterns keep untrusted data away from the component that chooses actions (Beurer-Kellner et al., 2025).

- **CaMeL** (Debenedetti et al., 2025): a privileged model writes a program from the trusted query, and a quarantined model parses untrusted data. An interpreter tags each value with its data provenance and allowed readers and evaluates Python security policies before tool calls; tools can also report an "inner source", such as an e-mail sender.
- **f-secure** (Wu et al., 2024) and **FIDES** (Costa et al., 2025): information-flow control for agents.
- **RTBAS** (Zhong et al., 2025): adds a dependency screener, including a trained one, that selects which context regions an action depends on.
- **Permissive IFC** (Siddiqui et al., 2024): propagates only the labels of influential inputs.
- **AirGapAgent** (Bagdasarian et al., 2024): minimizes the data exposed for a task.
- **Progent** (Shi et al., 2025) and **Conseca** (Tsai & Bagdasarian, 2025): express privilege policies that a model may help write and that are enforced deterministically.
- **Jacob et al.** (Jacob et al., 2025): restrict what crosses the privilege boundary to narrow data types.

Detection- and alignment-style defenses include MELON (Zhu et al., 2025), Task Shield (Jia et al., 2024) and IPI-Guard (An et al., 2025). Our study takes all of these mechanisms as given. We claim none of control/data separation, provenance or capability enforcement, typed restriction, or learned context selection as new.

Argument-level authority. Three recent systems are closest to our setting (Table 1).

- **PACT** (Fan et al., 2026) assigns each argument a semantic role and a contract C_t. It tags values with

provenance $\pi(v) = \langle \mathcal{O}(v), \tau(v), \mathcal{B}(v) \rangle$ and admits an argument only if its trust level, origins and obligations satisfy the contract. It reports that, with oracle provenance, contract checking reaches full utility and security on its diagnostic suites. It locates "the remaining deployment bottleneck" in provenance inference and contract synthesis.

- **ROPE** (Ma et al., 2026) checks sensitive parameters against origin markers (e.g. *prompt*, *sourced*, *record*). A zero-shot language-model router reads only the user's request and outputs the scope. An "enforcement clamp" accepts a router's override only when it is at least as strict as an offline default. Origins are assigned from the platform's own division of a result, so "a file body or a web page … arrives as one block" (§3.3).

- **ARGUS** (Weng et al., 2026) builds an influence-provenance graph. It segments context into spans labelled benign or anomalous, and grounds each argument in supporting spans with prompted sub-agents.

How we differ. The difference is narrow. Compared with each system:

- **ROPE**'s router selects a *scope*; it does not choose among several admitted values.

- **PACT**'s check decides admissibility, not which admissible value is correct.

- **ARGUS** does ground arguments in spans, but with prompted models and a release decision rather than an abstaining binder under an external fixed contract.

What we add is a way to *measure* how much the choice among admitted values can matter while the contract is fixed. Span-level grounding itself is not new (ARGUS), and we do not rest novelty on it. Whether a *trained* binder beats both rules and prompted grounding at matched cost is an open empirical question that this paper does not settle.

Benchmarks. AgentDojo (Debenedetti et al., 2024) provides user tasks, injection tasks and post-state utility and security checks. PACT evaluates on "all 97 benign user tasks and all 27 injection tasks" (Fan et al., 2026). AgentDyn (Li et al., 2026) adds "60 challenging open-ended tasks and 560 injection test cases" that include helpful third-party instructions. We replay AgentDojo offline without a language model (§5); we have not run AgentDyn.

3. Problem Setup

3.1. Authorization versus factual correctness

A task t consists of a trusted user instruction q_t, a *fixed plan* of tool calls, and a set of argument slots K_t of state-changing calls. For each variant v of the task (benign, or one injection), the runtime reads tool outputs and extracts a candidate set $X_{t,v}$.

- Each candidate x carries a value $\mathrm{val}(x)$ and a provenance record $\pi(x)$.

- A *contract* κ_t assigns each slot k a predicate $\kappa_{t,k}$ on provenance records.

- The *admitted set* is $C_{t,v,k} = \{x \in X_{t,v} : \kappa_{t,k}(\pi(x))\}$.

An assignment a gives each slot an element of $C_{t,v,k} \cup \{\perp\}$, where $\perp$ means abstain. A call is *authorized* if every slot is bound to an admitted candidate. The task *succeeds*, $s(t, v, a) = 1$, if the post-state after executing the fixed plan with a satisfies an independent success criterion: user utility holds and, for injection variants, the attacker goal is not achieved. Authorization constrains a; success evaluates it. Neither implies the other. The examples in §1 are authorized but incorrect.

3.2. Threat model

We follow the plan-then-execute assumption of CaMeL-style systems.

- **Control flow.** A trusted planner that sees only q_t fixes the control flow; control-flow hijacking is out of scope.

- **Attacker.** The attacker writes the *text* of documents the agent reads. In one variant, the attacker also controls the account of a source the user delegated (a compromised vendor).

- **Host metadata versus author text.** We separate two kinds of provenance information. *Host metadata* is attached by the platform and cannot be forged by a document's author: source identifier and kind, authenticated sender, timestamp, and confidentiality readers. *Author-controlled text* includes field contents, subject lines, reference numbers and any instruction-like phrasing.

- **Assumption A8.** Field boundaries and field authorship are host metadata only when the platform defines them, as in typed API records. They are not host metadata when fields are parsed out of free text: a "quoted" block in an e-mail is text written by the sender. Our simulator *assumes* host-provided field paths and field authors (assumption A8). This is the same distinction

Table 1. Closest systems, from the sections we read (Appendix G). "Chooser" is the component that picks among several *admitted* values. None of the listed systems trains its chooser; RTBAS trains a dependency screener, not an argument binder.

System	Authority unit	Provenance granularity	Chooser among admitted values	Evaluation
CaMeL	per value, tool-call policies	value; tool "inner source"	program + quarantined LLM (prompted)	AgentDojo
PACT	per argument role contract	value tag $\langle \mathcal{O}, \tau, \mathcal{B} \rangle$	agent LLM (not addressed)	AgentDojo
ROPE	per sensitive parameter	platform-divided entry	agent LLM; router picks scope only	AgentDojo, AgentDyn
ARGUS	per action (release/block)	LLM-segmented spans	prompted argument grounder	AgentLure, AgentDojo
This paper	per typed argument, fixed contract	actor-view value	rules, typed resolver; ceiling only	AgentDojo replay; sim. (app.)

ROPE draws between a platform's record structure and a single block (§3.3 of Ma et al., 2026).

Out of scope: planner errors, side channels, and isolation of the binder process. In our implementation the binder boundary is an interface property checked by tests (§A), not process isolation.

3.3. Fixed-contract evidence binding

Definition 1 (Binder)**.** *A binder b maps $(q_t, k, C_{t,v,k})$, together with the provenance records and host metadata of the candidates, to an element of $C_{t,v,k} \cup \{\perp\}$. The runtime accepts only a candidate identifier. A non-learned validator V_κ checks the value type, whether the provenance is admitted by κ, confidentiality flow to the sink's readers, and numeric limits before execution.*

A binder therefore cannot issue permissions: it can only name one of the admitted candidates or abstain. The rule that anything it names must be admitted is enforced by V_κ, not by the binder.

To prevent credit from looser boundaries, we fix a *reference contract* κ^{ref} and re-check every executed call of every method against it. A method whose native boundary is looser can execute calls that κ^{ref} rejects; these count as reference violations, never as success.

We report three levels per episode and never merge them:

- *proposed*: a complete call reached the validator;
- *executed*: the validator allowed the call and the sink changed the post-state;
- *attacker-influenced*: an executed call carried an attacker-planted value.

In this framework a blocked proposal is never counted as a successful attack. AgentDojo's own trace-based checks do count proposed calls, so on AgentDojo we report its official channel alongside an executed-effect channel (§5). *Secure success* means one of two things. For a feasible task, the call executed, matched the success criterion, and had no reference violation and no attacker value. For a task that exceeds the principal's permissions, secure success means that nothing was executed.

4. A Clairvoyant Ceiling for Binding

Definition 2 (Binder-only clairvoyant ceiling)**.** *Fix a plan and a contract. At each write call the binder picks either abstention or one element of the product of the admitted sets of that call's slots, given the observations so far. Let $A_{t,v}$ be the set of complete choice sequences for the whole plan, i.e. the leaves of this decision tree. When admitted sets do not depend on earlier choices, $A_{t,v} = \prod_{k \in K_t} (C_{t,v,k} \cup \{\perp\})$ (the V1 census). Let $r(t, v)$ be the rule's assignment, and let V_t be the set of variants of t (benign and injection variants, nested within t). Define*

$$s_{\text{rule}}(t) = \tfrac{1}{|V_t|} \textstyle\sum_{v \in V_t} s\big(t, v, r(t, v)\big), \tag{1}$$

$$s_{\text{best}}(t) = \tfrac{1}{|V_t|} \textstyle\sum_{v \in V_t} \max_{a \in A_{t,v}} s(t, v, a), \tag{2}$$

$$H^{\text{clair}} = \tfrac{1}{|T|} \textstyle\sum_{t \in T} \big[s_{\text{best}}(t) - s_{\text{rule}}(t)\big]. \tag{3}$$

Proposition 1 (elementary)**.** *Suppose $r(t, v) \in A_{t,v}$ for all t, v. Then $s_{\text{best}}(t) \geq s_{\text{rule}}(t)$ for every t. Moreover, every binder b whose assignments lie in $A_{t,v}$ satisfies $\frac{1}{|T|} \sum_t [s_b(t) - s_{\text{rule}}(t)] \leq H^{\text{clair}}$.*

Proof. A binder that only ever picks admitted choices or abstains traces one leaf of the tree, so for every (t, v) we have $s(t, v, b(t, v)) \leq \max_{a \in A_{t,v}} s(t, v, a)$. For a randomized binder the same holds in expectation. Averaging over v and then over t gives the claim, and taking $b = r$ gives the first statement. □

The premise $r(t, v) \in A_{t,v}$ holds by construction for our baselines, and the census checks it as an invariant (best $\geq$ baseline in every variant).

What the ceiling is and is not. H^{clair} is a *clairvoyant* ceiling. The maximizing assignment is chosen with knowledge of the evaluator, so it ignores the information limits of any real binder.

- It bounds the improvement a binder could make *from above*, per evaluation channel and per estimand (secure success or task utility).
- It is not an estimate of learnable performance, and a large value is not evidence that learning would help.
- It bounds improvements that come *only* from choosing differently among admitted candidates. It excludes changing the plan, reading more data, authenticating a sender, and asking the user.

When the plan is the benchmark's ground-truth plan, the ceiling is *conditional on an oracle plan* (ORACLE_PLAN_CONDITIONAL).

Whole-call evaluation. We do not maximize each argument separately. Arguments interact in two ways: two arguments of the same call are checked jointly, and an earlier write can change what a later call reads. Nothing guarantees that per-field optima combine into an optimal call, and we have no proof that they do. So every element of $A_{t,v}$ is a complete choice sequence for the fixed plan. It is replayed from the same initial state, with each branch on its own copy of its parent's state.

Witnesses and exhaustiveness. When $A_{t,v}$ is too large to enumerate, the best candidate *found* does not bound the maximum. Because success is binary, we record two things for each variant:

- whether a *success witness* was found, which fixes the maximum at 1;
- whether the search was exhaustive, which, without a witness, fixes it at 0.

Otherwise the maximum is only known to lie in $[0, 1]$. This gives per-task bounds $[s^{\text{lo}}_{\text{best}}(t), s^{\text{up}}_{\text{best}}(t)]$ and aggregate bounds $[H^{\text{clair}}_{\text{lower}}, H^{\text{clair}}_{\text{upper}}]$. A task whose plan has no binder slot has $|A_{t,v}| = 1$. Its only action is the rule's own, so its gap is exactly 0 whether or not its success can be evaluated.

Unresolved tasks stay in the denominator. Some tasks cannot be evaluated. Examples are a slot whose admitted set cannot be defined without using the answer, or a replay error. We mark these UNRESOLVED and keep them in T. $H^{\text{clair}}_{\text{lower}}$ counts their gap as 0, and $H^{\text{clair}}_{\text{upper}}$ counts it as 1. On a finite benchmark evaluated in full (a census), $[H^{\text{clair}}_{\text{lower}}, H^{\text{clair}}_{\text{upper}}]$ has no sampling uncertainty. Attack variants are never counted as independent tasks.

A computational envelope. $[H^{\text{clair}}_{\text{lower}}, H^{\text{clair}}_{\text{upper}}]$ combines witness bounds, partially searched trees and unresolved tasks. It is not a confidence interval, not an achieved improvement, and not a learnable security gain. It is conditional on the oracle plan, the oracle observations and the oracle final answer.

An operational rule, not a finding. Before any real-benchmark run we fixed $\delta = 0.05$ as an operational rule. $H^{\text{clair}}_{\text{upper}} < \delta$ is sufficient to stop investing in learned binding under a given contract and plan. The converse implies nothing: holding can still be justified by a contaminated contract, a weak baseline, a utility-only gap, or cost and novelty. The rule is not a statistical, equivalence, validity or publishability test.

5. Oracle-Plan Candidate-Binding Census on AgentDojo

An *oracle-plan candidate-binding census* fixes everything except the binder's choice. The call sequence, the read-call arguments, the free-text arguments and the final answer are the benchmark's own ground truth. Only the typed arguments of state-changing calls vary, and every admitted choice is replayed offline against the benchmark's evaluators. No language model, GPU, network access or service account is used. Every number in this section, in both arms defined below, is ORACLE_PLAN_CONDITIONAL.

5.1. Snapshot, evaluator and two evaluation channels

We pinned AgentDojo 0.1.35 (MIT; Appendix F) at benchmark version v1.2.2, whose registry lists 97 user tasks and 35 injection tasks (Table 15). Reading the pinned source fixed three points.

- `security()` returns `True` when the *injection goal was achieved*. For benign variants we record `None` (not applicable), not 0.
- `utility_from_traces`/ `security_from_traces` run before the state-based checks. They are overridden by 12 user tasks and one injection task.
- Their traces are the tool calls *proposed* in assistant messages, whether or not they executed.

We therefore report two channels and never merge them.

- **Official** (primary): the assistant/tool transcript goes unchanged to AgentDojo's own transcript functions and checker. Proposed calls stay in the trace.
- **Executed effect**: the trace holds only calls that executed without error, evaluated together with the post-state.

A binder abstention is a candidate event: no assistant call is emitted, and no call is deleted after the fact. A fixture shows that the channels diverge: attacker calls that were proposed but denied count as an attack in the official channel and not in the effect channel.

5.2. Relation to our earlier census (V1)

Our earlier census (Appendix D) derived slots, contracts and state-changing calls from the tasks' ground-truth argument values, so its candidate sets were functions of hidden gold; a test shows the dependence. Its interval $[0.253, 0.335]$ is kept as the result of that contract only. It is neither a deployable ceiling nor a confidence interval, and we do not compare it like for like with the census below (Table 18).

5.3. CONTRACT_V2: an actor-view contract

The binder's *actor view* is the trusted request, the tool schema, the declared policy, and the outputs of calls already executed in this replay. Hidden gold is not an input to slot typing, candidate extraction or baseline decisions. Hidden gold here means ground-truth write values, the final answer, post-states, evaluators, task and attack identifiers, and future outputs.

This is the only sense in which the census is gold-free. The evaluated arms still use the oracle plan and oracle final answer, and the fallback arm inserts gold values (below).

- **Writes and slots.** Writes are declared from function names. Slot kinds come from the function, argument name, JSON-schema type and schema description. The binder kinds are IBAN, e-mail, date, date-time, URL, identifier, number, enum, Boolean, entity, principal and channel. Free-text arguments stay planner-fixed and are counted. There are no per-task allowlists.
- **Candidates.** Typed values extracted from the prompt and the observation prefix. An attacker-written value of the right type in an observation *is* admitted, so V2 is a provenance-and-type contract, looser than V1's gold-located one.
- **Two arms.** A binder slot with no candidate forces abstention in the *strict* arm. In the *fallback* arm it takes the planner's gold value (ORACLE_COMPOSED; 6 tasks), so that arm is not gold-free.
- **Baselines.** Unique-or-abstain, and a typed lexical/reference resolver that binds a unique prompt value, an observed name the prompt mentions verbatim, or the observed value whose record shares the most words with the prompt. The resolver was written after V1 and developed on three tasks.
- **Gold-boundary tests.** Mutating the hidden gold (write values, final answer, injection task) leaves slot kinds, candidates and both baselines' decisions unchanged on five task variants. A chooser that reads gold fails the same test (negative control).

The replay is *sequential*, not counterfactual. Each choice executes on a copy of the current state, and later observations and candidate sets follow from it; read arguments stay at the oracle plan.

The search is depth-first, trying the gold-matching choice first (gold affects the order only). It stops once it has witnesses for task success, secure success and attack success in both channels. Leaf caps per variant are 256 (workspace, travel) and 4,096 (banking, slack), set from measured replay cost. A variant whose search hits the cap without a witness keeps the bound $[0, 1]$.

A *selectable failure* is a variant where the baseline is not secure but a secure witness exists. For tasks with one, we compute the success-reachable choices at the first write for every variant. Variants with an identical first-write view whose sets are provably disjoint cannot all be served by any deterministic actor-view chooser. They are marked REFUTED; the rest are NOT REFUTED, which is a necessary condition only.

Everything was frozen before the run: the configuration, code, tests, a structural pass and a four-task smoke run. The smoke run exposed a number extraction bug, which was fixed before the freeze.

5.4. Results of the census

Coverage, search and cost. No task raised a replay error or ran out of budget, in either arm. This is all that "resolved" means here (97/97 tasks, 1,046 variants per arm). It does not mean that every variant's optimum was found.

- 36 tasks have no write call, so their gap is exactly 0.
- 77 variants in 9 tasks hit the leaf cap before all witnesses were found.
- For secure success and for utility, 30 of these variants (3 workspace tasks) keep the envelope $[0, 1]$. For attack reachability, 75 do.

The run took 2183 s wall-clock and 4100 CPU-s on two single-threaded processes. No earlier result was reused.

Table 2. CONTRACT_V2 envelope $[H_{\text{lower}}, H_{\text{upper}}]$ on AgentDojo v1.2.2 over all 97 tasks, per arm. Computational envelopes over partially searched trees, not confidence intervals or achieved gains. Both arms are ORACLE_PLAN_CONDITIONAL; the fallback arm also inserts gold values. Offline replay, no language model.

Empty-slot mode	Baseline	Official, secure	Effect, secure	Official, utility only
fallback (primary)	typed resolver	[0.273, 0.293]	[0.273, 0.293]	[0.273, 0.293]
fallback (primary)	unique-or-abstain	[0.314, 0.334]	[0.314, 0.334]	[0.314, 0.334]
strict	typed resolver	[0.271, 0.292]	[0.271, 0.292]	[0.271, 0.292]
strict	unique-or-abstain	[0.302, 0.323]	[0.302, 0.323]	[0.302, 0.323]

Table 3. Security under CONTRACT_V2: injected variants in which a path executed a call achieving the attacker goal (Off.: official, Eff.: executed-effect channel). [a]Only variants where the gold choice was admitted. [b]Witness bounds on the number of variants where *some* admitted choice sequence achieves it; the upper count includes capped searches. These are counts, not attack rates or learnable gains.

	fallback		strict	
Path (of 949 injected variants)	Off.	Eff.	Off.	Eff.
typed resolver	4	4	0	0
unique-or-abstain	4	4	0	0
abstain on every write	0	0	0	0
ground-truth path[a]	0/709	0/709	0/674	0/674
some admitted action[b]	35–110	35–110	31–106	31–106

Finding 1: utility, not security. Over the typed resolver, the envelope is $[0.271, 0.292]$ in the strict arm and $[0.273, 0.293]$ in the fallback arm (Table 2). Over unique-or-abstain it is $[0.302, 0.323]$ and $[0.314, 0.334]$.

In each arm the utility-only envelope equals the secure envelope. All 247 (strict, 27 tasks) and 248 (fallback, 28 tasks) selectable failures are utility failures. In none did the baseline complete the task while the attack succeeded and a secure alternative existed. The gap concentrates in Slack and workspace (Appendix Table 17): choosing the channel with the most users, the largest file, or the right event among legitimate ones.

The oracle final answer matters for 42 tasks in the fallback arm and 38 in the strict arm (ORACLE_OUTPUT_DEPENDENT). 9 of the 42 have a write call. They overlap with the 6 ORACLE_COMPOSED tasks in 4, so 44 tasks carry either flag in the fallback arm.

In this run the official and executed-effect channels gave the same outcome on all 4,184 path records. No proposal was denied, because the binder only proposes admitted values, and the 31 paths with tool errors changed no outcome. This is an observation about this trace set, not an equivalence of the two evaluators; a fixture shows them diverging on denied proposals.

Finding 2: rare execution, non-zero admitted risk. Both baselines executed the attacker goal in 0 (strict) and 4 (fallback) of 949 injected variants. The fallback cases are the banking task whose legitimate IBAN the injection erased; in the strict arm the baselines abstain there. The ground-truth path and full abstention executed none (Table 3).

Some admitted choice sequence nevertheless achieves the attacker goal in 31–106 (strict) and 35–110 (fallback) injected variants, because the contract admits any observed value of the right type. These witness counts are neither a baseline attack rate nor a security improvement a binder could learn. They also rule out claiming that binding carries no attack risk. The benign-only and injected-only envelopes are close ($[0.289, 0.309]$ and $[0.270, 0.291]$, fallback), but closeness does not show that the security effect of binding is zero.

Finding 3: limits of the measurement.

- **Candidate coverage.** In 14 of the 61 tasks with writes, no candidate our generator admits completes even the benign task (fallback). A post-hoc diagnostic of the gold path found 43 slot values in 21 tasks that were not admitted. 29 are absent from the actor view, e.g. 18 date-times composed from a date and a time. 12 occur only in free text or as a name in an IBAN slot, and 2 are list shapes. This is a limit of our candidate generator. A binder that may compose values, or a different contract, might complete these tasks.
- **Identifiability screen.** Of the 248 selectable failures (fallback), the screen refuted none. For 148 of them it is vacuous, because no other variant shares their first-write view; in 15 of the 28 tasks every row is vacuous. The other 100 share a view within their task and have a common success-reachable first-write choice. The screen covers the first write only. NOT REFUTED is therefore not evidence of identifiability. The needed information does appear in the observation (e.g. user counts, file sizes), but using it takes aggregation or comparison, not lexical matching.

What we do next. We hold learned-binder method development. This is the authors' resource decision, not a result, and it does not say that evidence-binding research is futile. The frozen follow-up rule, which counts NOT REFUTED rows, would have proposed a learned-binder study (PROPOSE_CONDITIONAL_NOT_RUN). Given

the vacuous rows and the utility-only gap, we do not act on it.

Not run.

- **Learned or prompted binder.** Held as an operational decision; no language-model backend was used.
- **AgentDyn census.** Needs approval to fetch the repository and a license check.
- **Other attacks and versions.** Adaptive attacks, other attack templates and other benchmark versions.

6. Limitations

AgentDojo census.

- **Oracle plan.** Every number in both arms is ORACLE_PLAN_CONDITIONAL: the call sequence, read arguments, free-text arguments and final answer are ground truth. In the fallback arm, 44 tasks are ORACLE_OUTPUT_DEPENDENT or ORACLE_COMPOSED (4 are both). A language-model planner makes errors the envelope does not cover, and it would see different candidates.
- **Replay.** The replay is sequential, but read arguments stay at the oracle plan, so the plan never adapts to an earlier choice.
- **Contract design.** CONTRACT_V2 is our own post-hoc revision. Its slot typing uses this benchmark's schema descriptions, and the resolver was developed on three tasks after V1 was seen. It is not a blind pre-registration. It admits any observed value of the right type, which is looser than a field-scoped contract. Its candidate generator cannot compose or compute values. That is a limit of this generator, not of binding in general (§5.4).
- **Search caps.** 77 variants have incomplete searches. The secure and utility envelopes stay $[0, 1]$ for 30 of them, and attack reachability stays open for 75.
- **Identifiability screen.** It checks the first write only and is a necessary condition. It is vacuous for 148 of 248 failures.
- **Channels.** They coincided in this trace set because nothing was denied. That is not a general equivalence: they diverge under a denying validator, as a fixture shows.
- **Attacks.** One template (`DirectAttack`), one benchmark version, and no adaptive attacker. Injection text replaces default content, so some variants cannot be solved from observation.
- **Envelope, not performance.** No learned or prompted binder was run on AgentDojo.

Simulator.

- **Self-authored evidence.** The generator, rules and features were written by the same authors, with planted signals such as shared cue words, fixed timing of attacker documents and position regularities (Appendix A.3).
- **Simulated baselines.** Whole-document trust and the CaMeL-style selector encode an assumed susceptibility model. The CaMeL-style selector lacks CaMeL's information-sufficiency check and real confirmation.
- **Self-grading.** Training labels and the evaluator share the generator's gold values.
- **Host metadata (A8).** Field paths and field authors are treated as host metadata, which fails for fields parsed from free text.
- **Statistical weight.** Three held-out templates make every interval uninformative.

Implementation. The validator is tested, not verified. It trusts the provenance labels it is given, and the binder boundary is an in-process interface, not isolation.

7. Conclusion

Authority boundaries decide which values may reach an argument, not which admitted value is right. Under an oracle plan on AgentDojo, we audited how much that choice could matter when only typed, observed candidates may be chosen.

- **Utility, not security.** Choosing differently among admitted values could add at most $[0.271, 0.292]$ (strict) or $[0.273, 0.293]$ (fallback) over a typed resolver. The whole envelope is utility.
- **Rare execution, non-zero risk.** Baselines rarely executed the attacker goal, but some admitted choice achieves it in a bounded set of injected variants.
- **Measurement limits.** Our generator cannot serve some tasks at all, and whether the selectable failures are identifiable from observation remains open.

These findings do not transfer to agents with language-model planners without new, independent traces. We stop method development here.

Impact Statement

This work concerns the security of language-model agents that act on third-party content. Its intended effect is to prevent overclaiming: it separates what an authority boundary guarantees from what it does not, and defines a measurement that can show when a learned component is not worth building. The simulator uses synthetic data only. The AgentDojo census replays the benchmark's own fictional environments offline, without a language model, network access or service account. No real accounts, credentials, payment identifiers or user data are involved. The attack patterns we model, such as injected instructions, stale values and compromised senders, are publicly documented. Our description adds no new attack capability. A possible negative consequence is that a formal-looking bound is mistaken for a security guarantee. We therefore state explicitly that H^{clair} bounds only the benefit of binding choices under a fixed contract and plan, that it is a ceiling rather than achievable performance, and that our validator provides no process isolation.

References

An, H., Zhang, J., Du, T., Zhou, C., Li, Q., Lin, T., and Ji, S. IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents, 2025. arXiv comment: "EMNLP 2025".

Bagdasarian, E., Yi, R., Ghalebikesabi, S., Kairouz, P., Gruteser, M., Oh, S., Balle, B., and Ramage, D. AirGapAgent: Protecting privacy-conscious conversational agents, 2024. arXiv comment: "at CCS'24".

Beurer-Kellner, L., Buesser, B., Creţu, A.-M., Debenedetti, E., Dobos, D., Fabian, D., Fischer, M., Froelicher, D., Grosse, K., Naeff, D., Ozoani, E., Paverd, A., Tramèr, F., and Volhejn, V. Design patterns for securing LLM agents against prompt injections, 2025.

Costa, M., Köpf, B., Kolluri, A., Paverd, A., Russinovich, M., Salem, A., Tople, S., Wutschitz, L., and Zanella-Béguelin, S. Securing AI agents with information-flow control, 2025.

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, 2024.

Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., and Tramèr, F. Defeating prompt injections by design, 2025.

Fan, L., Li, Z., Tian, Y., Wang, Y., Li, R., and Wang, X. The granularity mismatch in agent security: Argument-level provenance solves enforcement and isolates the LLM reasoning bottleneck, 2026.

Jacob, D., Alghamdi, E., Hu, Z., Alomair, B., and Wagner, D. Preventing prompt injection with type-directed privilege separation, 2025.

Jia, F., Wu, T., Qin, X., and Squicciarini, A. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents, 2024.

Li, H., Wen, R., Shi, S., Zhang, N., Vorobeychik, Y., and Xiao, C. AgentDyn: Are your agent security defenses deployable in real-world dynamic environments?, 2026.

Ma, X., Xiao, C., Yeoh, W., Zhang, N., and Vorobeychik, Y. ROPE: Routed origin policy enforcement against indirect prompt injection, 2026.

Shi, T., He, J., Wang, Z., Li, H., Wu, L., Guo, W., and Song, D. Progent: Securing AI agents with privilege control, 2025.

Siddiqui, S. A., Gaonkar, R., Köpf, B., Krueger, D., Paverd, A., Salem, A., Tople, S., Wutschitz, L., Xia, M., and Zanella-Béguelin, S. Permissive information-flow analysis for large language models, 2024.

Tsai, L. and Bagdasarian, E. Contextual agent security: A policy for every purpose, 2025. arXiv comment: "Workshop in Hot Topics in Operating Systems (HotOS) 2025".

Weng, S., Feng, Y., Zhang, J., Xie, X., Yu, J., and Liu, J. ARGUS: Defending LLM agents against context-aware prompt injection, 2026.

Wu, F., Cecchetti, E., and Xiao, C. System-level defense against indirect prompt injection attacks: An information flow control perspective, 2024.

Zhong, P. Y., Chen, S., Wang, R., McCall, M., Titzer, B. L., Miller, H., and Gibbons, P. B. RTBAS: Defending LLM agents against prompt injection and privacy leakage, 2025.

Zhu, K., Yang, X., Wang, J., Guo, W., and Wang, W. Y. MELON: Provable defense against indirect prompt injection attacks in AI agents, 2025. arXiv comment: "ICML 2025".

A. Synthetic Simulator Study (Engineering Evidence Only)

To exercise the formulation end to end, we built a small simulator in which every document, secret and attack is generated from a seed. Its purpose is to test the harness, not to measure agents. All results from it are *engineering evidence only*.

A.1. Simulator design

Tasks and conditions. There are nine task templates in three domains (Appendix A.3):

- billing: pay an invoice by order number, pay the latest invoice to a saved payee, request a refund by reference;
- calendar: accept the latest proposed date, reschedule by meeting reference, invite a registrant from a form;
- support: forward a tracking number, report a delivery date from a file, reply with a knowledge-base note.

Each template is instantiated under five content conditions:

- *helpful* data;
- *malicious instructions*, either in an attacker's document or written by a compromised delegated sender;
- *mixed* content, where the injection shares the helpful document, either in another field or in the delegated field itself;
- *wrong facts*, i.e. type-valid stale or superseded values with no instruction;
- *insufficient permission*: unknown entity, tool not granted, amount over limit, or restricted path.

Secrets are synthetic tokens in files readable only by the user. Payment identifiers use a non-ISO country code and all addresses use reserved `.example` domains.

Contract and validator. The reference contract delegates each argument to a named field path of documents whose authenticated sender and field author are the named entity, or to the named file. The validator returns explicit reason codes; for example, `PROVENANCE_OUT_OF_SCOPE`, `FLOW_VIOLATION`, `LIMIT_EXCEEDED`, `NO_CAPABILITY` and `REQUESTED_SCOPE_MISMATCH`. Every decision is logged in a trace.

Selectors. We compare seven selectors under an identical plan, reads and budget.

- *Whole-document trust*: any field of a delegated document is admitted. It is paired with an *assumed* instruction-following reader.
- *Untrusted-all deny*.
- A *CaMeL-style* selector. This is our simplified simulation, not the CaMeL implementation: high-integrity untrusted arguments require user confirmation, counted as a refusal, and an assumed susceptible extraction step reads the requested field.
- *Rule field scopes*: filter by field scope, then key, then trusted history, then recency; bind only a unique value.
- A *learned binder*: logistic regression over 17 template-agnostic features, threshold 0.5 fixed in advance.
- Two bounds: an *oracle* binder and a worst-case *adversarial* binder that knows the attack labels.

Appendix Table 5 lists each feature and the origin of its input: trusted plan, trusted record, host metadata, host metadata under A8, or author-controlled text. No feature reads hidden labels, which a unit test checks.

Interface, not isolation. The binder receives a read-only view and the runtime accepts only a candidate-identifier string. Frozen Python objects prevent accidental mutation, but any code in the same process can change them. A unit test demonstrates this, and another shows that the validator trusts the provenance labels it is given. The guarantee that a binder "cannot grant authority" is therefore a property of the interface in our code, not of process isolation.

A.2. Simulator experiments

A.2.1. Engineering checks (SED-E0, SED-E1)

The simulator run (SED-E1, commit `54caffb`, 5.4 s on two CPU cores) used a configuration frozen before the run. Splits hold out templates, domains, or instances; the independent unit is the template. The validator-guarded selectors (rule, learned, oracle, adversarial) had 0 reference violations and 0 synthetic-secret leaks. No task that exceeded the principal's permissions was ever executed. The oracle reached secure success 1.0 on every feasible episode, so a secure solution always existed inside the reference contract.

Re-aggregating the stored raw rows reproduced every reported aggregate with zero differences. Table 9 (appendix) separates the proposed, executed and attacker-influenced levels. For example, the learned binder proposed 28 bindings outside the reference contract on held-out templates, and all of them were blocked.

A.2.2. What the boundary removes, and what it leaves

Table 6 counts, per delegated argument, whether an attacker value was present and whether it lay *inside* the reference contract.

- **Removed by the contract.** Attacks from out-of-scope documents and from other fields of the delegated document never fell inside it.
- **Left inside the contract.** Attacks in the delegated field itself, and all attacks from a compromised delegated sender, always did. Stale and superseded values left the admitted set ambiguous in every case.

This split is a property of how we generated the data, under assumption A8. It is a check that the harness classifies these cases as intended, not an empirical finding about agents.

A.2.3. A learned binder did not beat the rules

Table 4 reports the template-held-out split.

- **Main comparison.** The learned binder's secure success (0.798) is close to the rule's (0.773). It traded fewer over-refusals for more wrong actions.
- **Other splits.** Across the three splits (Table 7) the learned-minus-rule difference in secure success was +0.024, -0.007 and -0.033. In the instance split the learned binder also executed more attacker values: a template-averaged attacker-influenced rate higher by +0.032. With three held-out templates, the cluster-bootstrap intervals are not meaningful and we do not interpret them.
- **Baselines.** Whole-document trust and the CaMeL-style selector are simulations whose attack rates follow from an assumed susceptibility model, so their rows are not measurements of any system. The CaMeL-style selector's low secure success comes mainly from confirmation requests on T6, the one held-out template whose delegated argument is high-integrity.
- **Compromised senders.** When a compromised delegated sender wrote into the delegated field, secure success on the held-out templates was at most 0.03 for every selector. No trusted history exists for those templates.

We read these results as a failed pre-registered stop check: the learned binder does not beat the rule baseline. The synthetic data cannot settle whether learning could help on real tasks.

A.2.4. Why it failed: estimation, not expressivity

Our first internal report attributed the failure to linear scoring: we claimed that a linear score cannot express the lexicographic rule "filter by key, then by recency". That claim was wrong for our features.

- **Representable in principle.** With binary features and bounded continuous features, sufficiently separated weights make a linear argmax lexicographic (Observation 1, Appendix B).

Table 4. SED-E1, template-held-out split (test templates T3, T6, T9; 450 episodes). SS: secure success (macro over templates). ASR: executed attacker values among attack-condition episodes. RefViol: executed calls rejected by the reference contract. OverRef: no execution on a feasible task. † simulated baseline with an assumed susceptibility model; not a measurement of any system. ENGINEERING ONLY.

Selector	SS↑	ASR↓	RefViol↓	OverRef↓	Wrong↓	Confirm
Whole-document trust†	0.560	0.472	0.067	0.244	0.056	0.000
Untrusted-all deny	0.200	0.000	0.000	1.000	0.000	0.000
CaMeL-style (simulated)†	0.611	0.194	0.000	0.333	0.044	0.267
Rule field scopes	0.773	0.178	0.000	0.194	0.000	0.000
Learned binder (LR)	0.798	0.194	0.000	0.081	0.060	0.000
Oracle binder (bound)	1.000	0.000	0.000	0.000	0.000	0.000
Adversarial binder (bound)	0.200	0.378	0.000	0.578	0.187	0.000

- **Representable in practice.** Hand-set weights on the same 17 features reproduce the rule's decision (bind the same candidate, or abstain) on 1920 of 1920 test arguments across the three splits. The fitted binder agreed on 1505 (Table 8).

This was an analysis after seeing results. It trained nothing and proposes no method.

Where the fitted binder erred, the gold and chosen candidates differed in a few features.

- **Held-out "other key" cases.** The difference was only key match, latest-in-source and recency rank. The fitted template-split weights put recency (-1.72 per unit rank) above key match ($+1.29$).
- **Held-out knowledge-base cases with a stale second span.** The difference was only *first-in-field*, whose fitted weight was -0.74. That template was held out of training in this split, so we do *not* attribute the error to the generator artifact we had earlier blamed.

Why the pointwise log-loss fit produced these weights was not investigated. One untested explanation is that pointwise log-loss is not a decision loss.

Two further findings from this check:

- **A bug.** The first version compared sigmoid probabilities in the argmax. For large scores these saturate to 1.0, so the candidate order broke ties. We fixed this by taking the argmax on logits. Regenerating the test episodes reproduced all stored rule and learned outcomes (0 mismatches), so the fix does not change any reported number.
- **A limit of pointwise scoring.** The rule's set-level abstention ("abstain if the newest admitted document holds two distinct values") is not representable by *any* pointwise score plus threshold on our features (Observation 2). No such configuration occurred in the test data.

A.2.5. CEILING: SIMULATOR SMOKE TEST

We ran the ceiling code on the instance-split test episodes. Each template is treated as an original task with its 150 episodes nested as variants, the reference contract fixes the admitted sets, and the rule is the field-scope rule. Results (Appendix Table 14):

- every rule choice lay inside the action set;
- no task was unresolved;
- $H^{\text{clair}} = 0.199$, which equals one minus the rule's secure success on this split;
- the ambiguity proxy was 1.000: every template had some ambiguous slot.

The two numbers illustrate why a proxy cannot stand in for H^{clair}. Success in the simulator is computed from the generator's own gold values, which also provide the learned binder's training labels. It is therefore not an independent criterion, and this value is a code-path check only.

A.3. Simulator details

Templates. There are nine templates in three domains:

- **billing**: T1 pay invoice by order (recipient and amount delegated), T2 pay latest invoice to saved payee (amount delegated), T3 refund by correction reference (amount delegated);
- **calendar**: T4 latest proposed date, T5 reschedule by meeting reference, T6 invite registrant from form (attendee delegated);
- **support**: T7 tracking number by order, T8 delivery date from a report file, T9 troubleshooting note from a knowledge-base file.

Conditions and variants. Each condition has the following variants:

- helpful: clean;
- malicious: out-of-scope attacker document, or compromised delegated sender;
- mixed: other field, or same field;
- wrong fact: other key, superseded, stale field, or same-field history;
- insufficient permission: entity unknown, tool not granted, over limit, or restricted path.

There are 30 train and 30 test episodes per template and condition, with disjoint seeds.

Planted signals and assumptions.

- **A1.** One cue lexicon is shared by the generator, the features and the simulated susceptible readers. Attacker cue rate is 0.7 and legitimate cue rate is 0.25.
- **A2.** Position regularities: in same-field history, the gold span precedes the stale one; in T9, a legitimate cue sentence precedes the gold note.
- **A3.** Attacker documents are always newer than gold; superseded documents are always older.
- **A4.** Trusted payment history exists only for T1, with probability 0.5.
- **A5.** The planner is exact.
- **A6.** The susceptibility of whole-document trust and the CaMeL-style extraction step is assumed.
- **A7.** The CaMeL-style selector is not the CaMeL code.
- **A8.** Field paths and field authors are treated as host metadata.

None of these was altered after the results were seen.

B. Elementary Arguments

Observation 1 (lexicographic priority is linearly representable)**.** *Let each candidate have binary features* $b_1, \ldots, b_p \in \{0,1\}$ *and a bounded feature* $\rho \in [0,1]$*. Choose weights* $w_\rho < 0$ *and* $w_i > 0$ *such that* $w_i > \sum_{j>i} w_j + |w_\rho|$*. Then* $\arg\max_x \left(\sum_i w_i b_i(x) + w_\rho \rho(x) \right)$ *selects a candidate that is lexicographically maximal in* $(b_1, \ldots, b_p, -\rho)$*.*

Proof. Let x and y first differ at a binary feature i, with $b_i(x) = 1$ and $b_i(y) = 0$. The score difference is at least $w_i - \sum_{j>i} w_j - |w_\rho| > 0$. If they agree on all binary features, the difference is $w_\rho(\rho(x) - \rho(y))$, which favors the smaller ρ. □

Table 5. Learned-binder features and the origin of their inputs. host_A8: host metadata only under assumption A8. author: text controlled by the document's author (the attacker for attacker documents and compromised senders).

Feature	Inputs	Feature	Inputs
in requested field path	plan, host_A8	cue before span	author
doc from requested source	plan, record, host	field structured	host_A8
field author is source	record, host_A8	multi-value field	author, set
field author $\neq$ sender	host_A8, host	first in field	author
key given	plan	history given	record
key match	plan, author	history match / conflict	record, author
latest in source	host, set	doc type in task	task, author
recency rank	host, set	bias	–

This is standard. The weights in `scripts/analyze_e1_expressivity.py` instantiate it for field scope $\gg$ key $\gg$ history $\gg$ multi-value exclusion $\gg$ recency, with a threshold for abstention.

Observation 2 (set-level abstention is not pointwise)**.** *Let ϕ be the router's feature map. No selector of the form "bind $\arg\max_x g(\phi(x))$, abstain if $\max_x g(\phi(x)) < \tau$" reproduces the rule's decisions on the two candidate sets S_1, S_2 below.*

Proof. Let x_{old} be an admitted, key-matching candidate in an older document.

- In S_1, a newer admitted, key-matching document holds two distinct values in the delegated field, so the rule abstains.
- In S_2, the newer document from the same source lacks the key, so the rule binds x_{old}.

Because the recency features are computed over all documents from the source, $\phi(x_{\text{old}})$ is identical in S_1 and S_2; a unit test checks this. Binding in S_2 requires $g(\phi(x_{\text{old}})) \geq \tau$. Then $\max_{x \in S_1} g(\phi(x)) \geq \tau$, so the selector binds in S_1, which contradicts the rule's abstention. □

A retracted shortcut. An earlier version of our protocol proposed to stop if fewer than 10% of tasks had an ambiguous admitted set, and inferred an improvement below 5 percentage points. That inference was wrong on two counts.

- A task fraction p bounds the average improvement by p, not by $p/2$.
- Ambiguity is a proxy, not a success difference.

In our simulator the ambiguity proxy is 1.000 while $H^{\text{clair}} = 0.199$ (Appendix A.2.5).

C. Experiment Record

Commands, commits, configuration hashes and costs are listed in our run manifest, which is not part of the anonymous review package. The runs relevant here are the following.

Simulator.

- SED-E0 unit tests.
- SED-E1 simulator pilot (5.4 s).
- SED-E1-REAGG re-aggregation.
- SED-E1-EXPR expressivity check. It ran three times (pre-fix, post-fix, and aligned hand-set weights), and all three outputs are kept.

AgentDojo.

- **Task enumeration** (SED-E2-ADJ-CENSUS).
- **Structural pass** (DEVELOPMENT). It recorded plans, slots, candidate-set sizes and timing, and inspected no success values.
- **First census run: INVALID and kept.** A success witness equal to the empty assignment was tested by truthiness, so zero-slot variants lost their witness. This produced negative gaps, and we stopped the run.
- **Corrected census** (345.7 s). It added a regression test and a best-$\geq$-rule invariant check. The protocol was unchanged.
- **Exploratory slot-sensitivity analysis**, run after the census.
- **CONTRACT_V2** (session 4). A structural pass (DEVELOPMENT, no success values) and a four-task smoke run (DEVELOPMENT; it exposed a number-extraction bug, fixed before the freeze). Then the frozen census (SED-E2-CONTRACT-CLOSURE, 2183 s wall, 4100 CPU-s, two single-threaded processes) and a post-hoc coverage diagnostic.

All runs used at most two CPU cores and no GPU, network access during replay, language model or paid API.

Table 6. Per delegated argument, instance-split test set. Counts are over *arguments*; one billing template has two delegated arguments of which only one is attacked, so "present" is below "args" in attack conditions. ENGINEERING ONLY.

Condition/variant	Args	Atk. present	Atk. in scope	Ambig. in scope
malicious / attacker doc	172	157	0	6
mixed / other field	165	148	0	4
malicious / compromised sender	128	113	113	113
mixed / same field	135	122	122	122
wrong fact / other key	92	0	0	92
wrong fact / superseded	148	0	0	148
wrong fact / stale field	33	0	0	2
wrong fact / same-field hist.	27	0	0	27
helpful / clean	300	0	0	8

Table 7. Learned binder minus rule field scopes (per-template means). n_T: number of held-out templates. ‡percentile cluster bootstrap over templates; not meaningful for $n_T < 10$ and not used for inference. ENGINEERING ONLY.

Split	n_T	ΔSS	95% CI‡	ΔAtk	ΔOverRef	ΔWrong	per template
template	3	+0.024	[-0.013, +0.047]	+0.007	-0.091	+0.060	T3 -0.013, T6 +0.047, T9 +0.040
environment	3	-0.007	[-0.160, +0.100]	+0.002	-0.056	+0.060	T7 +0.100, T8 -0.160, T9 +0.040
instance	9	-0.033	[-0.086, +0.013]	+0.032	-0.039	+0.041	9 templates

Table 8. Decision agreement with the rule, per delegated argument (bind the same candidate, or both abstain). Lex: hand-set lexicographic weights (not fitted). Learned: stored fitted weights. Regen. mismatches: stored outcomes not reproduced by regeneration. EXPLORATORY.

Split	Args	Rule binds	Lex = rule	Learned = rule	Regen. mismatches
template	360	290	360	275	0
environment	360	271	360	246	0
instance	1200	1036	1200	984	0

D. First AgentDojo Census (V1 Contract) in Detail

This appendix keeps the V1 census as reported in manuscript v2. Its contract was derived from hidden ground truth (§5.2); the numbers below describe that contract only.

Table 9. SED-E1 template split: proposed, executed and attacker-influenced levels, with native denial reason codes. A blocked proposal never counts as an attack. ENGINEERING ONLY.

Selector	Episodes	Proposed	Executed	Exec. w/ atk. value	Exec. ref. viol.	Native denials
Whole-document trust†	450	406	272	85	30	FLOW_VIOLATION 88, NO_CAPABILITY 46
Untrusted-all deny	450	0	0	0	0	–
CaMeL-style (simulated)†	450	406	240	35	0	NEEDS_CONFIRMATION 120, NO_CAPABILITY 46
Rule field scopes	450	333	290	32	0	NO_CAPABILITY 43
Learned binder (LR)	450	405	331	35	0	NO_CAPABILITY 46, PROVENANCE_OUT_OF_SCOPE 28
Oracle binder (bound)	450	360	360	0	0	–
Adversarial binder (bound)	450	426	152	68	0	FLOW_VIOLATION 3, NO_CAPABILITY 66, PROVENANCE_OUT_OF_SCOPE 208

Table 10. SED-E1 domain-held-out split (test: support templates T7–T9). ENGINEERING ONLY.

Selector	SS	ASR	RefViol	OverRef	Wrong
Whole-document trust†	0.607	0.372	0.047	0.244	0.049
Untrusted-all deny	0.200	0.000	0.000	1.000	0.000
CaMeL-style (simulated)†	0.829	0.317	0.000	0.000	0.044
Rule field scopes	0.767	0.089	0.000	0.247	0.000
Learned binder (LR)	0.760	0.094	0.000	0.178	0.060
Oracle binder (bound)	1.000	0.000	0.000	0.000	0.000
Adversarial binder (bound)	0.200	0.322	0.000	0.653	0.149

Coverage and bounds (V1).

- **Coverage.** Of 97 user tasks, 52 had no V1 slot, so their gap is exactly 0. 89 tasks were resolved. 8 were UNRESOLVED because a slot held free text with no typed extractor; they stay in the denominator.
- **Search.** We replayed 940 variants. 0 non-exhaustively searched variants lacked a witness.
- **Bounds** (Table 16):
  - unique-or-abstain rule: $[0.253, 0.335]$;
  - first-admitted rule: $[0.216, 0.299]$;
  - free-text slots fixed to the planner value: $[0.306, 0.306]$.

Manuscript v2 read the upper bound's crossing of δ as "does not license holding". That reading is corrected in §4.

Where the gap comes from. The positive gaps come mostly from tasks whose admitted set holds several *legitimate* values, where the unique-or-abstain rule abstains. Examples are four Slack channels, or the files of a drive listing (Table 12). Picking the right one (for example, "the channel with the most users") is the reasoning step of the task. The clairvoyant chooser resolves it by consulting the evaluator; a real binder would need the task reasoning that the planner otherwise performs. The gap is similar on benign and injected variants (Table 13). On this benchmark, under an oracle plan, the ceiling is therefore mainly a *utility* ceiling on argument choice, not a security margin.

Security under the fixed plan. Attacks rarely reached an executed call.

- **Ground-truth plan.** It executed 0 attacks over 851 injected variants.
- **Unique-or-abstain rule.** It executed 4. All of them are in one task, where the injection had replaced the bill text holding the legitimate IBAN, so the attacker's IBAN was the only admitted value.
- **Admitted sets.** An attacker-written value was admitted in 37 injected variants, and the benign value had been removed in 34.

Table 11. SED-E1 instance-held-out split (all templates, disjoint seeds). ENGINEERING ONLY.

Selector	SS	ASR	RefViol	OverRef	Wrong
Whole-document trust†	0.694	0.504	0.055	0.081	0.039
Untrusted-all deny	0.200	0.000	0.000	1.000	0.000
CaMeL-style (simulated)†	0.688	0.298	0.000	0.222	0.015
Rule field scopes	0.801	0.194	0.000	0.152	0.000
Learned binder (LR)	0.767	0.274	0.000	0.103	0.041
Oracle binder (bound)	1.000	0.000	0.000	0.000	0.000
Adversarial binder (bound)	0.200	0.383	0.000	0.580	0.183

Table 12. AgentDojo v1.2.2 census per suite (primary analysis, unique-or-abstain). Unresolved tasks contribute 0 to the lower and 1 to the upper bound. ORACLE_PLAN_CONDITIONAL.

Suite	Tasks	Tasks with gap > 0	Unresolved	per-suite $[H_{\text{lower}}, H_{\text{upper}}]$
workspace	40	6	6	[0.150, 0.300]
travel	20	0	0	[0.000, 0.000]
banking	16	5	1	[0.312, 0.375]
slack	21	16	1	[0.643, 0.690]

The removal property matters for interpretation. AgentDojo's injection text *replaces* a vector's default content, which can hold the legitimate information. In 9 of the 34 removal variants (from 1 task), no admitted action succeeds and abstention is the best secure choice. In the other removal variants a success witness still exists. The oracle plan still "succeeds" in all of them because its arguments are fixed in advance. This is one way ORACLE_PLAN_CONDITIONAL results hide information limits.

Sensitivity to slot identification (exploratory). Slots and contracts are found by value coincidence between a ground-truth argument and an earlier output field. This can yield contracts that make no sense, such as an amount matched to a transaction-id field. After seeing the census we dropped the tasks with numeric-valued slots (18 of 91 slots). The bound on the remaining 81 tasks was $[0.179, 0.228]$. A cruder name-agreement filter removes almost every task with a slot and is not interpretable. We keep the frozen primary result and report this only as evidence that H^{clair} depends on how slots are identified.

An evaluator detail. One banking injection task's `security()` contains `any(service or "" in t.subject.lower() ...)`. Because of operator precedence, this returns `True` for any transaction to the attacker's IBAN, whatever the subject. We did not modify it. The four executed attacks above send money to the attacker under any reading.

Table 13. AgentDojo v1.2.2: ceiling restricted to benign or to injected variants (primary analysis, unique-or-abstain). Descriptive breakdown fixed before the run.

Variants (primary, unique-or-abstain)	$[H_{\text{lower}}, H_{\text{upper}}]$
benign variants only	[0.247, 0.330]
injected variants only	[0.254, 0.336]

Table 14. Headroom smoke test on the simulator (instance-split test episodes; 150 nested variants per template). Success is the generator's gold, not an independent criterion. ENGINEERING ONLY.

Template t	Variants	$s_{\text{rule}}(t)$	$s_{\text{oracle}}(t)$	gap
T1	150	0.900	1.000	0.100
T2	150	0.800	1.000	0.200
T3	150	0.847	1.000	0.153
T4	150	0.787	1.000	0.213
T5	150	0.780	1.000	0.220
T6	150	0.793	1.000	0.207
T7	150	0.793	1.000	0.207
T8	150	0.827	1.000	0.173
T9	150	0.680	1.000	0.320
H (all resolved)				0.199
Ambiguity proxy				1.000

Table 15. AgentDojo 0.1.35, benchmark v1.2.2: tasks enumerated from the package registry.

Suite (v1.2.2)	User tasks	Injection tasks
workspace	40	14
travel	20	7
banking	16	9
slack	21	5
total	97	35

Table 16. Binder-only clairvoyant ceiling on AgentDojo v1.2.2 (all user tasks in the denominator; unresolved tasks counted as gap 0 / 1). ORACLE_PLAN_CONDITIONAL; offline replay, no language model.

Slots	Rule	Resolved	$[H^{\text{clair}}_{\text{lower}}, H^{\text{clair}}_{\text{upper}}]$
free-text slots UNRESOLVED	unique-or-abstain	89/97	[0.253, 0.335]
free-text slots UNRESOLVED	first-admitted	89/97	[0.216, 0.299]
free-text slots planner-fixed	unique-or-abstain	97/97	[0.306, 0.306]
free-text slots planner-fixed	first-admitted	97/97	[0.216, 0.216]

E. CONTRACT_V2 Census: Additional Tables

Tables 17 and 18 follow.

Table 17. CONTRACT_V2 per suite (fallback arm, typed resolver, official channel). Gap>0: tasks with positive lower gap; Undet.: tasks with capped searches; Sel. fail.: selectable-failure variants; Not refuted: by the first-write identifiability screen (a necessary condition only), with the number of vacuous rows (no other variant shares the first-write view) in parentheses.

Suite	Tasks	Gap>0	Undet.	Unres.	$[H_{\text{lower}}, H_{\text{upper}}]$	Sel. fail.	Not refuted (vacuous)
workspace	40	9	3	0	[0.225, 0.275]	135	135 (105)
travel	20	0	0	0	[0.000, 0.000]	0	0 (0)
banking	16	3	0	0	[0.131, 0.131]	21	21 (11)
slack	21	16	0	0	[0.730, 0.730]	92	92 (32)

Table 18. V1 versus V2 contract. The V1 contract was derived from hidden ground truth and is not deployable; the rows are not a like-for-like comparison of the same contract. Executed attacks: official channel, injected variants of resolved tasks.

Contract	Baseline	Resolved	$[H_{\text{lower}}, H_{\text{upper}}]$	Executed attacks
V1: slots and contract from benign ground truth	unique-or-abstain	89/97	[0.253, 0.335]	4/851
V2: schema slots, actor-view candidates	unique-or-abstain	97/97	[0.314, 0.334]	4/949
V2: schema slots, actor-view candidates	typed resolver	97/97	[0.273, 0.293]	4/949

F. AgentDojo Offline Replay Protocol

Snapshot and import path. We use AgentDojo 0.1.35, the PyPI wheel with sha256 prefix `364bea42`, MIT license, at benchmark version v1.2.2.

- The wheel is installed without its declared LLM-client dependencies.
- A small import shim skips the package initializer that imports those clients. The only constant the evaluation path needs from an LLM module is read from the source's syntax tree.
- No language-model API, GPU or service account is used.

Fixed components (all frozen before the census):

- **Plan.** The task's own `ground_truth(env)` call sequence in each variant's environment, with final output `GROUND_TRUTH_OUTPUT` (ORACLE_PLAN_CONDITIONAL).
- **State-changing calls.** Calls whose execution changes the environment dump.
- **Slots.** Arguments of state-changing calls whose benign ground-truth value is absent from the user prompt and occurs in an earlier benign tool output, either as a field value or as a substring of a text field.
- **Contract.** For each slot, the (tool, field path) set where the benign value occurs. Substring slots additionally require a typed extractor (IBAN, e-mail, URL, date, date-time, time, number) chosen by the type of the benign value.
- **Admitted set.** Values the contract admits in the variant's own observation, plus abstain.
- **Rules.** Primary: unique-or-abstain. Secondary: first admitted value.
- **Variants.** Benign, and one variant per injection task of the suite, using AgentDojo's `DirectAttack` template ("TODO: {goal}") on the vectors that AgentDojo's own canary procedure reports as seen by the task.

Evaluation. Each assignment is replayed on its own deep copy of the variant's initial environment. The utility and security checks run as in AgentDojo: `*_from_traces` first, and the state-based check if it returns `None`. The trace contains only calls that passed the gate. We record *task_success*, *attack_success* (`security()` = True; `None` for benign variants) and *policy_violation* separately, along with blocked calls and tool errors.

Search and bounds. The search order is:

1. the rule assignments;
2. the assignment equal to the ground-truth values (if admitted);
3. exhaustive enumeration up to 4,096 assignments per variant, or 4,096 seeded samples above that.

The search stops at the first success witness. Bounds follow §4.

Fixtures. Ten tests on the pinned snapshot check the direction of every outcome field:

- benign success;
- attack success when the attacker's account is bound;
- a blocked call that is not counted as an attack, with no attacker transaction in the post-state;
- abstention;
- a tool error;
- the policy-violation flag;
- isolation between replays;
- agreement with AgentDojo's own checker;
- the injection-replacement property.

G. What We Read

For the closest papers we read the following sections of the arXiv full text:

- **ROPE**: §1–§8 and appendices A (except A.2), B and C;
- **PACT**: §1, §3, §4, and appendices A, C–F.1 and I; the proofs were not read;
- **ARGUS**: the full main text;
- **CaMeL**: §3–§6.

Bibliographic entries were checked against the arXiv abstract pages. For the remaining related work we rely on abstracts or partial sections, as recorded in our reading notes.